> Basado en: docs/icp.md | Tipo: producto (SaaS) | Supuestos abiertos: 3 | Fecha: 2026-09-08

ReleaseView — Solución

1. Qué es

ReleaseView es un panel interno de gestión de versiones, features y despliegues que centraliza, cliente por cliente, qué versión y qué features tiene activos cada uno de los clientes de un producto SaaS o agente de IA desplegado en múltiples instalaciones. Resuelve la dispersión de esa información entre herramientas de desarrollo (Azure DevOps, Jira) y la memoria de una sola persona (el líder técnico), para que cualquiera del equipo pueda responder de inmediato preguntas de negocio sobre el estado de cada cliente.

2. La promesa / resultado

De: el PM/PO tiene que decir "te confirmo después" cuando un cliente pregunta por su versión o features, y pierde 1-2 horas cruzando información entre herramientas antes de cada release.

A: responde en el momento consultando ReleaseView, y arma la lista de "qué cliente recibe qué" en minutos, sin depender de que el líder técnico esté disponible.

3. Para quién es (y para quién no)

Para quién es: PM, Product Owner o líder técnico en una consultora de IT o empresa de software que gestiona un SaaS o agente de IA desplegado en múltiples clientes con versiones/features divergentes, usando Azure DevOps o Jira como herramienta de desarrollo.

Para quién no es:

- Equipos que gestionan un solo cliente o una instalación única del producto.
- Empresas con un producto interno sin variantes entre usuarios.
- Equipos donde el mismo desarrollador construye, despliega y conoce el estado de cada cliente de memoria.
- Organizaciones que ya tienen una herramienta de release management funcionando.
- *(Estimado)* Equipos que no usan Azure DevOps ni Jira — hoy la integración parte de ahí.

4. Qué hace / capacidades

- **Ver de un vistazo qué versión y qué features tiene activos cada cliente** — la capacidad núcleo, resuelve el JTBD funcional priorizado del ICP.
- **Responder en el momento** cuando un cliente pregunta por su estado, sin decir "te confirmo después".
- **Armar la lista de "qué cliente recibe qué"** antes de un release, sin cruzar herramientas a mano.
- **Identificar qué clientes todavía no recibieron un módulo/feature específico.**
- **Registrar manualmente lo que Azure DevOps/Jira no rastrea** — por ejemplo, qué features de negocio están habilitados comercialmente para cada cliente, algo que no siempre es 1:1 con un ticket cerrado.

5. Cómo funciona

Alta: se conecta la cuenta de Azure DevOps o Jira (integración de lectura) → ReleaseView trae automáticamente lo que puede (releases, tickets cerrados, etiquetas de versión) → el equipo completa a mano lo que no está ahí (ej. qué features de negocio están habilitados por cliente, o el estado real de un despliegue si no quedó registrado en la herramienta de desarrollo).

Uso diario: el PM/PO/líder técnico consulta el panel por cliente (¿qué tiene activo?) o por feature (¿qué clientes no lo tienen todavía?) y responde en el momento.

Antes de un release: filtra qué clientes corresponden a la próxima versión y arma la lista de despliegue desde el panel, en vez de cruzar Azure DevOps + correos + chats a mano.

6. Qué obtenés / entregables

- Acceso al panel para el equipo interno.
- Integración activa de lectura con Azure DevOps y/o Jira.
- Vista por cliente: versión activa + features habilitados + historial de despliegues.
- Vista por feature: qué clientes lo tienen y cuáles no.

7. Formato y modelo de entrega

SaaS self-serve para el uso diario, con **setup asistido** en el alta (conectar la integración con Azure DevOps/Jira suele requerir permisos y una configuración inicial que conviene acompañar). Soporte técnico estándar por los canales habituales — sin SLA formal en esta etapa.

8. Requisitos y costo de herramientas

- Cuenta de Azure DevOps o Jira ya existente en la organización (no es un costo nuevo — ya lo tienen).
- Permisos de lectura sobre el proyecto correspondiente, para habilitar la integración.
- Navegador — sin instalación local.
- *(Estimado)* si más adelante se suman más fuentes (ej. otro tracker), se evalúa caso por caso.

9. Precio base

Modelo de cobro: suscripción mensual por organización/equipo (no por asiento, no por cantidad de clientes gestionados) — un precio fijo independiente de cuántos clientes finales se administren adentro.

Precio ancla: *(Supuesto abierto — a definir con* `/costo` *una vez calculada la infraestructura y con más claridad de mercado).*

Supuestos abiertos

🔴 **Alcance real de la integración con Azure DevOps/Jira:** se definió como híbrida (automática + carga manual) pero no está mapeado en detalle qué campos trae cada integración vs. qué queda 100% manual — se precisa en `/arquitectura`.

🔴 **Exclusión de equipos sin Azure DevOps/Jira:** es una inferencia razonable (la integración parte de ahí), no confirmada explícitamente por el owner como límite duro del producto.

🔴 **Precio ancla sin definir:** el modelo de cobro (suscripción por organización) está decidido, pero el número queda pendiente hasta correr `/costo`.